

Similarity Index of Cellular Automata

Erick Jesse Angeles López¹[0009–0001–0218–2341] and Sukanta Das²

¹ Instituto Politécnico Nacional, Escuela Superior de Cómputo, ALIROB Laboratory, Ciudad de México, México

angeleslopezerickjesse@gmail.com

² Indian Institute of Engineering Science and Technology, Shibpur, Howrah, West Bengal, India

Abstract. TODO: abstract.

Keywords: Cellular automata · Elementary cellular automata · Similarity index · Transition graph · Spectral moments · Attractor basins.

1 Introduction

Cellular automata are complex systems: functions that define behaviors that are often chaotic but at other times easily predictable. Many studies have proposed classifications to tell different behaviors apart, such as Wolfram’s classification, TODO, or the set of life-like rules. These classifications are usually designed according to behavior, but how similar are the rules within them? And how different are they from the rules of another set?

At present, classifications are the only measure we can use to decide whether two rules are alike, apart from directly comparing their exact behavior. However, there is no measure that tells us how similar they are. The purpose of this research is to define a method to compare the behavior of any two rules and measure their degree of similarity.

To study these similarities, it is first necessary to identify which exact relations already exist between different domains, find the equivalences, and use them as a reference metric for the proposed algorithms, which essentially reduce to computing spectral moments together with a method that captures the growth of the attractor mass.

2 Preliminaries

Cellular automata are a collection of cells arranged in a lattice, where each cell takes a state from a finite set. The system evolves in discrete time steps and, at each step, all cells update their state synchronously as a function of the states of their neighboring cells [5,6,4].

Definition 1. A cellular automaton is a 4-tuple $(\mathcal{L}, \mathcal{S}, \mathcal{N}, \mathcal{R})$ where:

- $\mathcal{L} \subseteq \mathbb{Z}^D$ is the D -dimensional cellular space. Each cell is identified by a vector $\vec{i} \in \mathcal{L}$ that represents its position in the space.

This work is done as a project in the Indian Summer School on Cellular Automata 2026.

- $\mathcal{S} = \{0, 1, \dots, k-1\}$ is the finite set of k states. The state of cell $\vec{i}$ at time t is denoted by $\sigma_{\vec{i}}(t) \in \mathcal{S}$.
- $\mathcal{N} = (\vec{v}_1, \vec{v}_2, \dots, \vec{v}_m)$ is the ordered tuple of the m displacement vectors that define the neighborhood. The neighbors of cell $\vec{i}$ are located at positions $\vec{i} + \vec{v}_j$, with $j = 1, \dots, m$.
- $\mathcal{R} : \mathcal{S}^m \rightarrow \mathcal{S}$ is the local rule, which assigns the new state of a cell from the states of its m neighbors:

$$\sigma_{\vec{i}}(t+1) = \mathcal{R}(\sigma_{\vec{i}+\vec{v}_1}(t), \dots, \sigma_{\vec{i}+\vec{v}_m}(t)).$$

The order of the tuple $\mathcal{N}$ matters: it determines the numbering of the rules and allows us to define relations such as mirror rules (Section 3.1).

2.1 Finite space and boundary conditions

Ideally, cellular automata evolve in infinite spaces; in practice, however, boundaries are imposed that restrict the cellular space [1,3]. The boundary condition can be open (a fixed state is assigned to the missing neighbors) or periodic (the missing neighbors of a cell on the boundary are the cells on the opposite boundary) [3,1].

Throughout this work we always use periodic boundaries. In one dimension, the space is a ring of n cells, $\mathcal{L} = \mathbb{Z}_n$.

2.2 Neighborhoods and rules

For one-dimensional cellular automata with radius r , the neighborhood contains every cell at distance at most r , including the cell itself ($m = 2r + 1$), and the local rule is written as

$$\sigma_i(t+1) = \mathcal{R}(\sigma_{i-r}(t), \dots, \sigma_i(t), \dots, \sigma_{i+r}(t)), \quad \mathcal{R} : \mathcal{S}^{2r+1} \rightarrow \mathcal{S}.$$

The elementary cellular automaton (ECA) has $k = 2$, radius $r = 1$ ($m = 3$) and rule $\mathcal{R} : \{0, 1\}^3 \rightarrow \{0, 1\}$. There are $k^m = 2^3 = 8$ possible neighborhood patterns and each one is assigned a new state, so the total number of rules is $k^{k^m} = 2^8 = 256$. Each rule is identified by a decimal number obtained by reading, as a binary number, the states assigned to the patterns 111, 110, $\dots$, 000.

2.3 Global function and transition graph

Let $C = \mathcal{S}^{\mathcal{L}}$ be the set of configurations, with $N = |C| = k^{|\mathcal{L}|}$. The global function $F : C \rightarrow C$ applies the local rule to all cells synchronously:

$$F(\sigma)_{\vec{i}} = \mathcal{R}(\sigma_{\vec{i}+\vec{v}_1}, \dots, \sigma_{\vec{i}+\vec{v}_m}), \quad \forall \vec{i} \in \mathcal{L}.$$

The transition graph $G_F = (C, E)$ has one node per configuration and a directed edge $\sigma \rightarrow F(\sigma)$. Since F is a function, every node has out-degree 1 (a functional graph). Each connected component contains exactly one cycle, the attractor, from which transient trees hang and form its basin. For each $x \in C$ we define its period $p(x)$ as the length of the cycle it reaches, and its height $h(x)$ as the minimum number of steps needed to reach it ($h(x) = 0$ if x is periodic). Configurations without a predecessor are known as Garden of Eden configurations.

2.4 Quotient by translations

Let T be the translation by one position in $\mathcal{L}$ (a rotation of the ring $\mathbb{Z}_n$). With periodic boundaries, the global function commutes with the translation, $F \circ T = T \circ F$, so two configurations that differ by a translation have equivalent trajectories. This allows us to build the quotient graph, whose nodes are the classes of configurations under translation and whose edges are $[x] \rightarrow [F(x)]$. All configurations in the same class have the same height, and the number of nodes drops to approximately $N/|\mathcal{L}|$. When proportions are computed on the quotient graph, each class is weighted by its size.

3 Exact relations: Inheritance properties

This method arises as an extension of the well-known equivalence criteria: state permutation and mirror rules. We suggest that there are additional relations between rules of different spaces. For example, rule 0 always has the same behavior regardless of the number of states, the size of the neighborhood or the dimension. This raises the following question: what are the similarity criteria between rules of different spaces?

To this end, we call the domain of a rule the 3-tuple of dimension, states and neighborhood on which it is defined. Two rules belong to the same domain if they share all three elements.

3.1 Rules in the same domain

To compare the behavior of rules in the same domain we can apply state permutation and mirror rules, which in ECA reduce the 256 rules to 88 classes [2].

Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two cellular automata with the same domain, and let $(x_1, \dots, x_m) \in \mathcal{S}^m$ be the state of the neighborhood. We say that $\mathcal{A}_1$ and $\mathcal{A}_2$ are equivalent if one of the following conditions holds:

- **State permutation:** there is a bijection of states $\pi : \mathcal{S} \rightarrow \mathcal{S}$ such that

$$\mathcal{R}_2(x_1, \dots, x_m) = \pi \left(\mathcal{R}_1 \left(\pi^{-1}(x_1), \dots, \pi^{-1}(x_m) \right) \right), \quad \forall (x_1, \dots, x_m) \in \mathcal{S}^m.$$

That is, if we relabel all the states, the structure remains the same. For $\mathcal{S} = \{0, 1\}$, the only non-trivial one is $\pi(x) = 1 - x$, which simply swaps the two states.

- **Mirror rules:** let $M : \mathcal{S}^m \rightarrow \mathcal{S}^m$ be the reflection of the neighborhood, $M(x_1, \dots, x_m) = (x_m, \dots, x_1)$. We say that $\mathcal{A}_2$ is the mirror rule of $\mathcal{A}_1$ if

$$\mathcal{R}_2(x_1, \dots, x_m) = \mathcal{R}_1(M(x_1, \dots, x_m)).$$

That is, if we reflect the initial configuration, evolve it with $\mathcal{A}_1$ and reflect the result, we obtain exactly the evolution of $\mathcal{A}_2$.

3.2 State reduction

Let $\mathcal{A}$ be a rule. State reduction is possible when there are two states $a \neq b \in \mathcal{S}$ that satisfy the following conditions:

1. State a is never produced by the rule: $\mathcal{R}(x_1, \dots, x_m) \neq a, \forall (x_1, \dots, x_m) \in \mathcal{S}^m$.
2. State a behaves like b in every position of the neighborhood: $\mathcal{R}(x_1, \dots, a, \dots, x_m) = \mathcal{R}(x_1, \dots, b, \dots, x_m)$.

Then the reduced automaton is $\mathcal{A}' = (\mathcal{L}, \mathcal{S}', \mathcal{N}, \mathcal{R}')$ with $\mathcal{S}' = \mathcal{S} \setminus \{a\}$.

The justification is as follows: since state a is never produced, it can only appear in the initial condition, and since its transitions coincide with those of b , both have the same behavior for every configuration σ . Therefore, from the first step on, the evolution of $\mathcal{A}$ is exactly that of $\mathcal{A}'$.

3.3 Neighborhood reduction

Neighborhood reduction is possible when the state of a neighbor does not affect the transition, that is, there is an index j such that

$$\mathcal{R}(x_1, \dots, x_j, \dots, x_m) = \mathcal{R}(x_1, \dots, x'_j, \dots, x_m), \quad \forall (x_1, \dots, x_m) \in \mathcal{S}^m, \forall x'_j \in \mathcal{S}.$$

Then the reduced automaton is $\mathcal{A}' = (\mathcal{L}, \mathcal{S}, \mathcal{N}', \mathcal{R}')$ with $\mathcal{N}' = \mathcal{N} \setminus \{\vec{v}_j\}$.

If the removed neighbor is not at one of the ends, the neighborhood is adjusted so that it is contiguous with $m - 1$ cells. For the purposes of this research, this equivalence is applied to any neighbor, under the hypothesis that changing the position of the neighbors only transforms the space of the evolution and not the behavior. With this relation, ECA drops to 81 classes, a range we consider acceptable.

3.4 Equivalence classes

With these four relations (state permutation, mirror, state reduction and neighborhood reduction) it is possible to relate rules from different domains. Applying them iteratively builds a relation graph whose connected components are the equivalence classes. Although this classification is still binary (two rules either are or are not equivalent), it serves as a reference to validate the following methods: if two rules are equivalent, their similarity index should be one.

4 Structural comparison of transition graphs

The relations obtained in Section 3 depend only on the rule vector. As an alternative, we propose to compare the global dynamics: we define a finite periodic space of n cells, which contains $N = |\mathcal{S}|^n$ configurations. By evolving each of them, we obtain the transition graph G_F of the rule.

As a parallel method, all the translations of a configuration (Section 2.4) are considered a single node, which yields a quotient graph with approximately $|\mathcal{S}|^n/n$ nodes.

We then say that two rules have the same behavior if their transition graphs have the same topology, that is, if they are isomorphic.

4.1 Isomorphism and canonical form

The isomorphism comparison identifies exactly the rules related by state permutation and mirror rules, which confirms that the transition graph captures the similarity between rules.

This process is computationally expensive. As an alternative, we use the canonical form of the transition graph: a topological encoding of a graph that does not depend on the node labels, so that two graphs are isomorphic if and only if their canonical forms are equal. The canonical form is built with Algorithm 1.

Algorithm 1 Canonical form of a transition diagram

```

1: Add every Garden of Eden node to a queue  $Q$ 
2: while  $Q$  is not empty do
3:   Pop  $v$  from  $Q$  and assign it as code the sorted list of the codes of its predecessors
4:   Add the code of  $v$  to the predecessors of its successor  $s(v)$ 
5:   if  $s(v)$  has no unprocessed predecessors then
6:     Add  $s(v)$  to  $Q$ 
7:   end if
8: end while
9: Encode each cycle (nodes that never entered  $Q$ ) as its lexicographically smallest rotation
10: return the sorted list of the codes of all cycles

```

These methods have three main restrictions:

1. The comparison is binary.
2. The comparison is between rules of the same domain.
3. The result depends on the size n of the space (see Figure 1).

In ECA, the number of classes grows with n up to a limit that oscillates between 75 and 88 behaviors (Figure 1):

Full: At least 4 cells are needed to distinguish 82 behaviors and 6 for the maximum of 85. The groupings depend on whether the space is even or odd. From $n = 8$ on, some patterns appear: even n groups $\{15,170\}$, $\{23,178\}$ and $\{77,232\}$, n multiple of 4 groups $\{105,150\}$, odd n groups $\{10,34\}$, $\{24,46\}$ and $\{136,160\}$, and $n = 7$ and $n = 23$ group $\{60,90\}$. Combining two sizes greater than 5 (one even and one odd) yields the 88 classes. They are also obtained with $n = 3$ and $n = 4$.

Quotient: At least 6 cells are needed to distinguish 83 behaviors and 9 for the maximum of 85. From $n = 7$ on, the number depends only on $n \bmod 4$: 80 classes if n is odd, 85 if $n \equiv 2 \pmod{4}$ and 84 if $4 \mid n$. For every $n \geq 6$ it groups $\{12,34\}$, $\{15,51\}$ and $\{170,204\}$.

The partitions of the full graph and of the quotient are not nested. With odd n , the grouping of the full graph also occurs in the quotient, but with even n each one separates

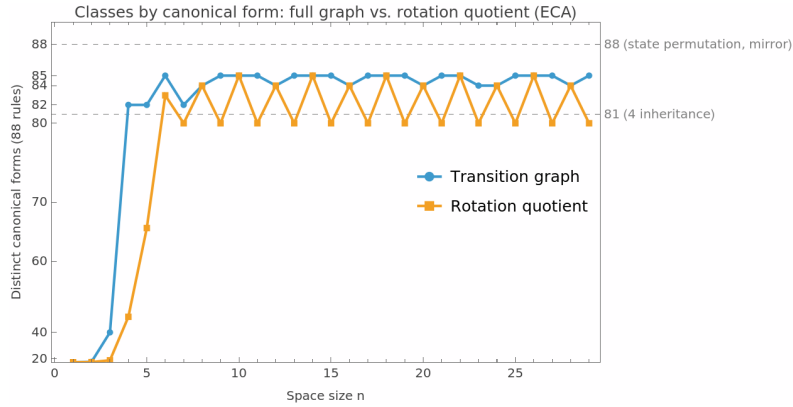


Fig. 1: Number of canonical forms among the 88 classes with the full transition graph and the quotient by translations (the y axis is on a cubic scale).

pairs that the other groups. Therefore, using both canonical forms on the same space distinguishes the 88 behaviors with $n = 6, 10, 12, 14, 18, 22, 24$ and 26 . That is, a single space of 6 cells is enough if both the full graph and its quotient are used (Table 1).

n	1	2	3	4	5	6	7	8	9	10	11	12
Full	3	9	40	82	82	85	82	84	85	85	85	84
Quotient	3	6	16	46	66	83	80	84	80	85	80	84
Both	3	10	40	85	82	88	84	87	85	88	85	88

Table 1: Number of classes with the canonical form of the full graph, of the quotient, and of both.

5 Spectral methods

The methods seen so far only detect equality between rules. To obtain a gradual measure, we represent each rule as a numerical vector (embedding) derived from its transition graph, so that the distance between vectors works as a similarity index.

5.1 Eigenvalues

In this method we compute the eigenvalues of the adjacency matrix A of the transition graph, sort them and use them as the embedding of the rule: two rules are compared through the cosine distance between their vectors. If two graphs are isomorphic, their adjacency matrices are similar through a permutation and produce the same eigenvalues, so the distance is zero. Otherwise, the distance measures how different their spectra are. This method obtains a maximum of 74 distinct behaviors (with 18 cells) in ECA.

This method has two limitations:

Blindness to transients: Each connected component has a basin made of transient trees. Since the trees only contribute zero eigenvalues, the spectrum depends only on the attractors and not on the trees. ECA rules 0 and 8 are not related by any inheritance property; however, both have a single fixed-point attractor made of all zeros, reached in 1 step with rule 0 and in at most 2 steps with rule 8, so their eigenvalues are identical.

Domain dependence: The number of eigenvalues equals the number of nodes $N = |S|^n$, so only rules with the same number of states and the same space size can be compared.

In the full graph the number of classes oscillates between 67 and 74 from $n = 8$ on, with the maximum at $n = 18$ (Figure 2). For every size, the partition given by the canonical form is finer than the one given by the spectrum. The quotient yields fewer classes (between 56 and 63 from $n = 12$ on) because, besides the blindness to transients, it identifies each rule with its composition with the translation, such as $\{15,51\}$ and $\{170,204\}$. The quotient adds no information to the spectrum: combining the 29 sizes of both graphs reaches a limit of 77.

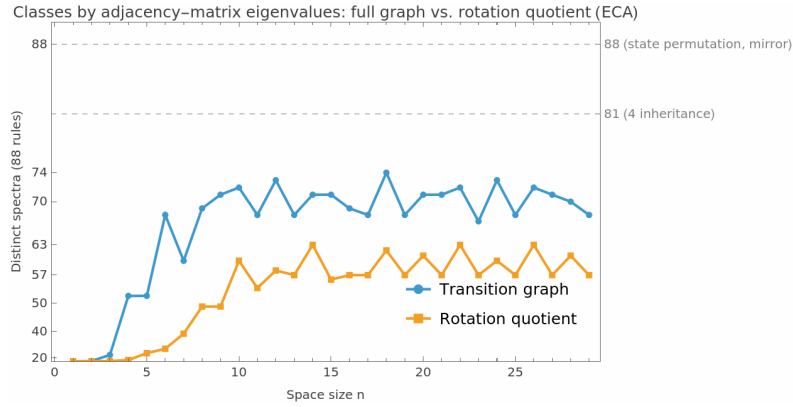


Fig. 2: Number of distinct spectra among the 88 representative rules with the full transition graph and with the quotient graph by translations.

The groups $\{0,8\}$, $\{1,19\}$, $\{2,24,46\}$, $\{4,12\}$, $\{18,126\}$, $\{36,72\}$, $\{128,136\}$, $\{130,152\}$, $\{132,140\}$ and $\{162,168\}$ have the same spectrum for every size computed. They are rules with the same cycles that differ only in their transients.

5.2 Spectral moments

To remove the domain dependence we propose to use spectral moments. Instead of computing the full spectrum, for each step $k = 1, \dots, K$ we measure the proportion of configurations that return to themselves after k steps:

$$m_k = \frac{|\{x \in C : F^k(x) = x\}|}{N}.$$

The length of the vector does not depend on the number of nodes but on the window size K , so rules with different numbers of states, neighbors or spaces can be compared. The drawback is that K must be chosen so that the relevant cycles of both graphs fall inside the window. The vectors are compared through the cosine distance, which is our similarity index.

In ECA, with the quotient graph by rotations of 14 cells and $K = 10$, we obtain 63 distinct vectors for the 256 rules. However, since m_k depends only on the eigenvalues, this method inherits the blindness to transients (Figure 3).

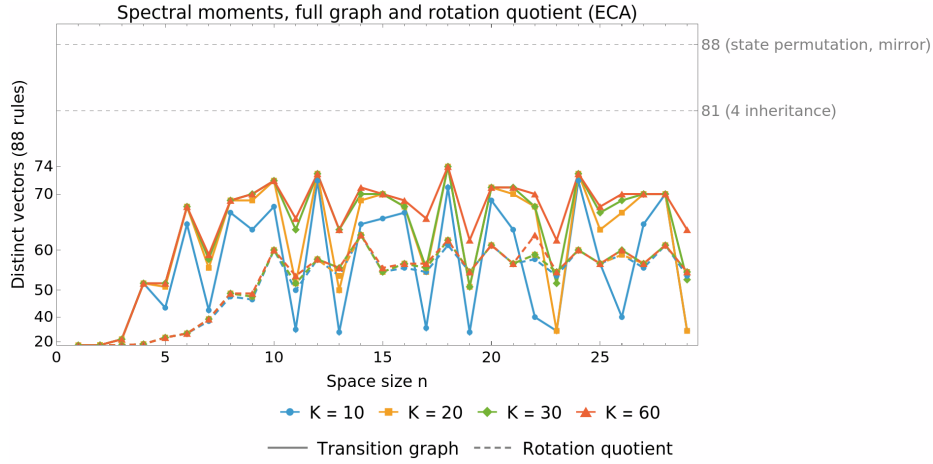


Fig. 3: Number of distinct spectral moment vectors among the 88 representative rules for $n = 1, \dots, 29$ and $K = 10, 20, 30, 60$, in the full graph (solid line) and in the quotient graph (dashed line).

A cycle of length $L > K$ is not captured by the vector. In the full graph with $K = 10$, the sizes with long cycles lose information ($n = 11, 13, 17, 19, 23$ and 29 give between 31 and 34 classes). The sizes with many divisors give up to 72 ($n = 12$ and 24).

In the quotient the cycles become shorter and K barely matters: with $K = 10$ it finds between 54 and 63 classes from $n = 13$ on, with a maximum of 63 at $n = 14$.

Combining the 29 sizes gives 77 classes in the full graph and 68 in the quotient, the same as with the eigenvalues, for any K .

5.3 Spectral moments with attractor mass

To distinguish rules such as 0 and 8, we need to measure, besides the attractors, how strongly they attract the other configurations, that is, to quantify how many configurations reach each attractor and in how many steps. The idea is that each cycle is still activated every p steps, as in the spectral moments, but at each activation we also add the configurations that have already reached it.

Using the height $h(x)$ and the period $p(x)$ of each configuration, we define

$$\tilde{m}_k = \frac{|\{x \in C : p(x) \mid k \text{ and } h(x) \leq k - p(x)\}|}{N}, \quad k = 1, \dots, K.$$

That is, a configuration counts at step k if its attractor is activated at that step and the configuration had already reached it before the previous activation. Thus, cycle nodes ($h = 0$) count from the first activation ($k = p$), nodes with $1 \leq h \leq p$ from the second ($k = 2p$), nodes with $p < h \leq 2p$ from the third, and so on.

Since every periodic configuration satisfies the condition when $p \mid k$, the vector decomposes as

$$\tilde{m}_k = m_k + t_k,$$

where m_k is the spectral moment and t_k is the proportion of transient configurations that count at step k . Therefore, the new vector extends the spectral moments with the information of the basins, and keeps the advantage of having a length K independent of the domain.

For example, with 5 cells ($N = 32$), rule 0 sends every configuration to zero in one step ($h \leq 1$), whereas in rule 8 each block of ones leaves an isolated 1 that disappears in the next step ($h \leq 2$); of the 32 configurations, 12 reach zero in at most one step. Their vectors are

$$\tilde{m}^{(0)} = \frac{1}{32} (1, 32, 32, 32, \dots), \quad \tilde{m}^{(8)} = \frac{1}{32} (1, 12, 32, 32, \dots),$$

which are no longer equal, even though their spectral moments are.

This method has some limitations. The resolution with which the height is measured depends on the period, since configurations are grouped in intervals of p steps: with long periods, a configuration with $h = 1$ and another with $h = p$ count at the same step. Moreover, a cycle with $p > K$ is never activated.

With the attractor mass the number of classes increases considerably (Figure 4). In the full graph, the sizes with many divisors give between 80 and 83 classes with $K = 10$, against a maximum of 74 with the eigenvalues and 72 with the spectral moments. The sizes with long cycles still depend on the window ($n = 29$ gives 52 classes with $K = 10$ and 81 with $K = 60$). In the quotient the number of classes depends only on $n \bmod 4$ from $n = 7$ on: with $K = 60$ we obtain 74 classes if n is odd, 80 if $n \equiv 2 \pmod{4}$ (79 with $n = 18$) and 76 if $4 \mid n$.

Rules 0 and 8 are separated for every $n \geq 3$. Combining the 29 sizes gives 86 classes in the full graph, and two small sizes are enough (for example, $n = 3$ and $n = 6$). Only two pairs remain together. In $\{1, 19\}$ the exact heights are different (with $p = 2$, some configurations have $h = 1$ in one rule and $h = 2$ in the other), but they fall in the same interval of p steps: this is the resolution limitation described above. In $\{4, 12\}$ the joint distribution of height and period is identical and the rules differ only in the shape of the trees, which only the canonical form distinguishes. The quotient, on the other hand, groups the rules that differ by a composition with the translation ($\{15, 51\}$, $\{170, 204\}$, $\{4, 12, 34\}$, $\dots$) and only reaches 81 classes. Using the full graph and the quotient on the same size gives 87 classes with $n = 6, 10, 12, 14, 18, 22, 24$ and 26.

Figure 5 and Table 2 compare all the methods. The canonical form leads in the number of classes, but the spectral moments with attractor mass are the second best

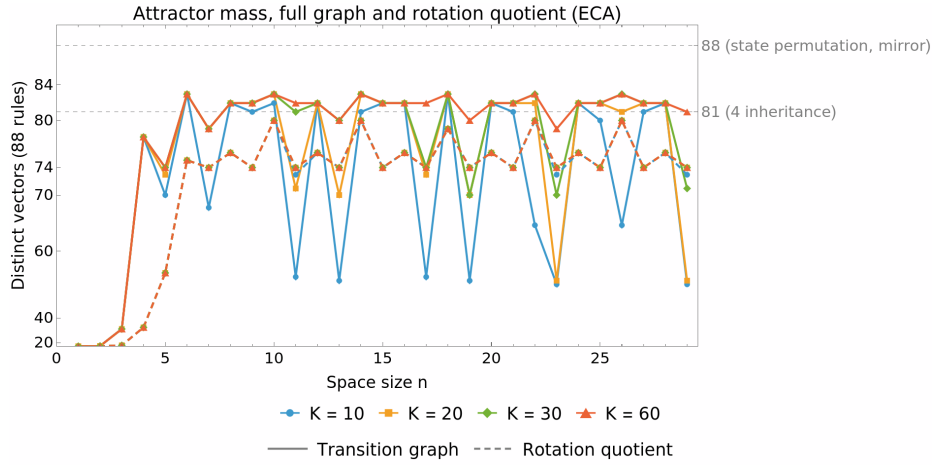


Fig. 4: Number of distinct spectral moment vectors with attractor mass among the 88 representative rules for $n = 1, \dots, 29$ and $K = 10, 20, 30, 60$, in the full graph (solid line) and in the quotient graph (dashed line).

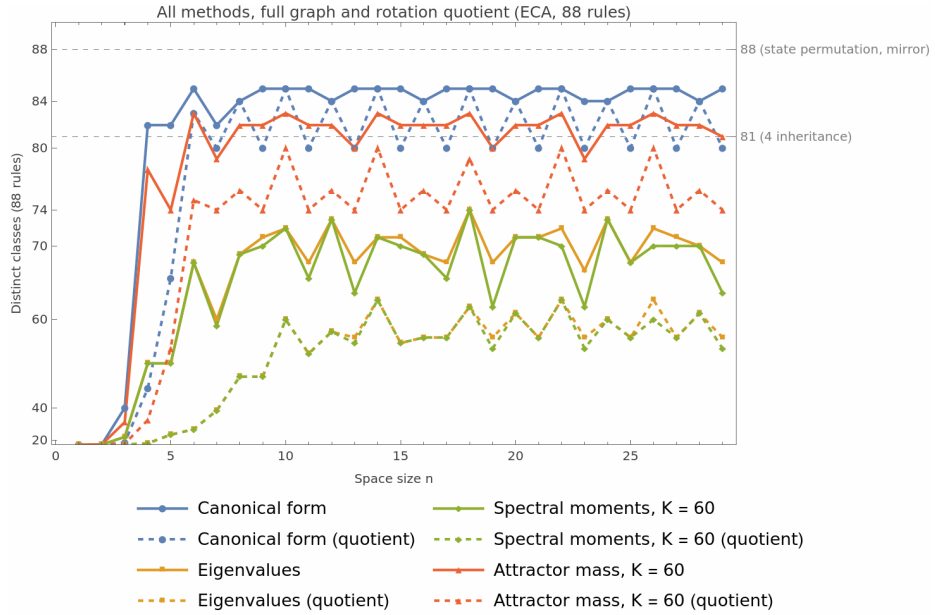


Fig. 5: Number of classes among the 88 representative rules with each method for $n = 1, \dots, 29$.

method, producing up to 87 classes in total, followed by the eigenvalues and the plain spectral moments.

Method	Full graph		Quotient		Full
	One size	All	One size	All	+ quotient
Canonical form	85	88	85	85	88
Eigenvalues	74	77	63	68	77
Spectral moments	74	77	63	68	77
Attractor mass	83	86	80	81	87

Table 2: Maximum number of classes among the 88 representative rules. “One size”: best n ; “All”: combines the 29 sizes; “Full + quotient”: both graphs.

Acknowledgments. TODO: acknowledgments.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Bhattacharjee, K.: Cellular Automata: Reversibility, Semi-Reversibility and Randomness. Ph.D. thesis, Indian Institute of Engineering Science and Technology, Shibpur (2019), <https://arxiv.org/abs/1911.03609>, last accessed 2025/08/23
2. Martinez, G.J.: A note on elementary cellular automata classification. *Journal of Cellular Automata* **8** (2013)
3. Ureña Ponce, O.: Diseño de un procesador de autómatas celulares para la simulación de sistemas complejos. Master’s thesis, Universidad de Guadalajara, Guadalajara, México (2015), <https://riudg.udg.mx/bitstream/20.500.12104/79915/1/MCUCEI01068FT.pdf>
4. Sancho Caparrini, F.: Autómatas celulares. Universidad de Sevilla (2016), https://www.cs.us.es/~fsancho/Blog/posts/Automatas_Celulares.md.html, last accessed 2025/09/24
5. Weisstein, E.W.: Cellular automaton. *MathWorld – A Wolfram Resource* (2002), <https://mathworld.wolfram.com/CellularAutomaton.html>, last accessed 2025/07/23
6. Wolfram, S.: Cellular automata as simple self-organizing systems. Technical Report CALT-68-938, California Institute of Technology (1982), <https://content.wolfram.com/sw-publications/2020/07/cellular-automata-self-organizing-systems.pdf>